Proyecto Práctico: Desarrollo de Aplicación Asistida por Agentes con OpenCode, Supabase y Looker Studio

1. Descripción del Proyecto

Diseñar, estructurar e implementar una aplicación funcional en equipo utilizando el asistente **OpenCode**. Cada equipo tiene libertad para definir la temática del proyecto y el stack tecnológico (Frontend/Backend/Móvil), asegurando el cumplimiento estricto de la arquitectura de agentes, la persistencia en la nube, el control de acceso y el análisis de datos.

2. Requisitos Técnicos y Funcionales

A. Entorno de Desarrollo y Agentes de IA (OpenCode)

- **Arquitectura de Agentes:** Diseñar y configurar un esquema de agentes especializados dentro de OpenCode adaptados a las necesidades del proyecto (por ejemplo: arquitecto de software, frontend engineer, backend engineer, QA/tester).
- **Skills:** Instalar y configurar las *skills* pertinentes para OpenCode según el stack tecnológico elegido.
- **Integración MCP (Model Context Protocol):** Integrar servidores MCP en el entorno de desarrollo si el flujo de trabajo lo requiere (por ejemplo: MCP para base de datos, sistema de archivos o herramientas externas).
- **Homogeneidad del Entorno: Todos** los integrantes del equipo deben replicar y mantener exactamente la misma estructura de carpetas, configuraciones de OpenCode, definiciones de agentes y variables de entorno (.env.example).

B. Persistencia y Autenticación

● **Base de Datos:** Implementación sobre **Supabase** (PostgreSQL). Debe incluir un modelo relacional normalizado con sus respectivas tablas, claves foráneas e índices. ● **Autenticación y Seguridad:** Sistema de registro e inicio de sesión funcional implementado mediante **JWT (JSON Web Tokens)**. Las rutas o recursos sensibles deben estar protegidos y validar el token en cada petición.

C. Analítica y Visualización de Datos

- **Dashboard en Looker Studio (Data Studio):** Conectar la base de datos de Supabase (o un dataset derivado) a Google Looker Studio.
- **Métricas:** Construir un panel interactivo con al menos:
    - ○ 3 KPIs principales del negocio/aplicación.
    - ○ Gráficos temporales, comparativos y de distribución.
    - ○ Filtros dinámicos (fechas, categorías o estados).

D. Control de Versiones y Metodología de Trabajo

● **Repositorio en GitHub:** Todo el código fuente, esquemas y documentación deben estar alojados en un repositorio público o privado con acceso al docente. ● **Estrategia de Ramas (GitFlow):** Es obligatorio el uso estricto de GitFlow: ○ main / master: Código en producción, estable.

- ○ develop: Integración de características terminadas.
- ○ feature/*: Desarrollo de módulos o funcionalidades específicas.
- ○ release/* y hotfix/*: Estabilización y corrección de bugs críticos.

● Se evaluará el historial de commits, la participación equitativa de todos los integrantes y el uso de *Pull Requests* con revisión de código.

3. Estructura de Entregables en el Repositorio El

repositorio debe contener en la raíz un archivo README.md detallado que incluya:

1. **Descripción de la solución:** Problema que resuelve y público objetivo.
2. **Stack tecnológico:** Lenguajes, frameworks y dependencias principales.
3. **Esquema de Agentes y Skills:** Documentación del diseño de agentes implementado en OpenCode, lista de *skills* utilizadas y justificación de servidores MCP (si aplica).
4. **Instrucciones de instalación:** Pasos exactos para levantar el proyecto en local y replicar el entorno de OpenCode de forma homogénea.
5. **Diccionario de datos / Modelo ER:** Diagrama de la base de datos en Supabase.
6. **Enlace al Dashboard:** Enlace con permisos de visualización al reporte de Looker Studio.

4. Modalidad de Entrega

● **Formato de entrega en Classroom:** Enlace al repositorio de GitHub y enlace al dashboard de Looker Studio.